

## CS3 Project 2: Connect Four (Spring 2026)

### Project description

In this project, we'll write a program for the game Connect Four and have a tournament between our automated players. For information on the game, see [http://en.wikipedia.org/wiki/Connect\\_Four](http://en.wikipedia.org/wiki/Connect_Four).

### Getting started

The project code will be organized into one top-level folder (which we open in vscode), with three subfolders containing Java source files for the corresponding packages. Two of the subfolders, connectFour and sample, should be copied from the starter code, here:

<https://drive.google.com/drive/folders/1EOWhSbkHcShGsF7VUMeX-jtkMMU2Goe?usp=sharing>. The third subfolder should be your first name in lower case, which is where you'll put your own Java files. Note every Java file in a folder/package must start with a package declaration.

The connectFour folder/package has two Java interfaces:

- Grid declares two public methods (plus a few constants):
  - int getPlayerAt(int row, int col) returns the player number (1, 2, or 0 for empty) at the given row and column. Note the bottom row has index 0.
  - boolean isColumnFull(int col) returns true if the given column is full, otherwise false.
- Player declares two public methods:
  - int getMoveColumn(Grid g) returns the column index (0 through 6) in which the player decides to move, based on the given grid.
  - String getPlayerName() returns the player's name (for human players this should be the name of the person playing, and for automated players it should include the name of the person who wrote that player's code, like "ArthurBot").

The sample folder/package has one Java class that shows how to put a class into a package, implement the Player interface, and use the Grid interface; you'll also use it as one of the choices of player in your game:

- SampleRandomPlayer implements the connectFour.Player interface:
  - int getMoveColumn(Grid g) returns a random nonempty column index
  - String getPlayerName() returns "SampleRandom"

### The Board class

Make a Board class that represents the state of your board and implements the Grid interface:

1. It must use private attributes for the board state, including an int[][].
2. It must have a public Board() constructor that initializes an empty board.
3. It must implement both Grid methods (getPlayerAt and isColumnFull)
4. It must have a public void playInColumn(int col, int player) method.
5. It must have a public int classifyPosition() method, which returns 1 if player 1 has four in a row, 2 if player 2 has four in a row, 0 if the game is a draw, and -1 if the game is not over.
6. It must override public String toString() to return a human-friendly String representation of the position, as shown in the example transcript near the end of this document.

## The Game class

Make a Game class that includes a main method and any private methods you decide will make your code easier to write and read. For example, consider writing a private static Player choosePlayer(int playerNumber) method that prompts the user to choose a player (random, human, automated) for that player number and returns a new instance of the corresponding class.

Your program must first prompt the user to choose the types of players (random, human, automated), construct instances of the corresponding Player classes, and store those objects in variables of type Player. Then, you must construct a new instance of your Board class and print it to show the empty board. Then in a loop, alternate getting each player's move (by calling their getMoveColumn methods), printing a one-line message like Player 1 moved in column 3, checking for invalid move (out of range or in a full column) and treating illegal moves as loss by forfeit, then if the move is valid calling your board's playInColumn method to make it, printing the new board position, then calling your board's classifyPosition method to check whether the game is over and if so print an appropriate message and return, otherwise switch the current player so the game will continue normally.

## The HumanPlayer class

Make a HumanPlayer class that implements the Player interface:

1. It must have two private attributes: Scanner scanner, String playerName.
2. It must have a public HumanPlayer(Scanner scanner, int playerNumber) constructor that initializes all attributes, including prompting the user for that player's name.
3. Its getMoveColumn method should prompt the user to enter a move column number (0-6) in a loop until the user enters a valid move, and then return that move.
4. Its getPlayerName method should just return its playerName attribute value.

## The AutomatedPlayer class

Make an AutomatedPlayer class that implements the Player interface:

1. It must have a public AutomatedPlayer() constructor.
2. The string returned by the getPlayerName method must include your name.
3. The getMoveColumn method must only return valid moves (non-full column 0 through 6) for valid positions, must move reasonably quickly (one second or less per move on a standard laptop), and for full credit must play much better than SampleRandomPlayer.
  - a. Reminder: the bottom row (where the first move in a column goes) has index 0.
  - b. For almost any interesting strategy, your getMoveColumn method will need to figure out whether it is player 1 or player 2; one way to do that is to count the number of nonzero player numbers with the grid's getPlayerAt method – if that number is even, you're player 1, otherwise you're player 2.
  - c. Note: a fairly simple strategy like "if I can win immediately, do that, otherwise if I can stop my opponent from winning immediately, do that, otherwise play in a random column" will play much better than SampleRandomPlayer, however it probably won't win the class tournament.

## Collaboration

You may help each other with design, testing, and debugging, however you should not share code. Also, the tournament will be more interesting with different approaches to the automated players.

## More suggestions and optional elements

Add a second public constructor to your Board class that takes a Grid as a parameter, so your automated player can convert the current Grid into a temporary instance of your own Board class and then use your Board methods to do things like make candidate moves and classify the resulting positions. You could also add more public methods to your Board class like `int playerNumberToMove()`, to avoid looping over the entire grid in your automated player code.

When checking for four in a row, carefully consider both players and each of the four possible directions: horizontal, vertical, diagonal up, diagonal down.

Consider Making a TestPlayer class that takes an `int[]` as a constructor parameter and makes those moves in sequence without even looking at the grid, so you can test your Board and Game classes by playing TestPlayer instances against each other that you construct to check various four-in-a-row situations as well as illegal moves and draws. You can of course do the same kind of testing with HumanPlayer, but it's nice to have tests that you can keep running automatically until they all pass.

If you're not sure how to start, try watching the TwentyOneExampleVideo.mp4 in the starter project folder, which shows how to do a much simpler project with similar structure, or ask in class.

## What to turn in (as a shared link to a Google drive folder with the following files)

Include all the files from your package subfolder plus your reflection document (at least these):

- Board.java
- Game.java
- HumanPlayer.java
- AutomatedPlayer.java
- Reflection: One paragraph about your experience on the project, and one paragraph about your approach to making your automated player play well.

## Grading rubric (25 points total)

- Readable code following Java conventions, use of `int[][]`, thoughtful reflection: 5 points
- Board class, managing state, detecting wins and draws, `toString`: 5 points
- Game class, picking players, playing game, reporting moves and result: 5 points
- HumanPlayer class, managing name, prompting until move is legal, returning move: 5 points
- AutomatedPlayer class, `getPlayerName`, legal moves, plays quickly, plays well: 5 points

### Example start and end of an interactive game (user input in *italics*):

Please choose player 1 (random, human, automated): *random*  
Please choose player 2 (random, human, automated): *human*  
Player 2, please enter your name: *Arthur*  
Starting game between player 1 (SampleRandom) and player 2 (Arthur)

|     |  |   |  |   |  |   |  |   |  |   |  |   |  |   |  |
|-----|--|---|--|---|--|---|--|---|--|---|--|---|--|---|--|
| 5   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 4   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 3   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 2   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 1   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 0   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| --- |  |   |  |   |  |   |  |   |  |   |  |   |  |   |  |
|     |  | 0 |  | 1 |  | 2 |  | 3 |  | 4 |  | 5 |  | 6 |  |

It is player 1's move

Move 1: player 1 moved in column 1

Current position:

|     |  |   |  |   |  |   |  |   |  |   |  |   |  |   |  |
|-----|--|---|--|---|--|---|--|---|--|---|--|---|--|---|--|
| 5   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 4   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 3   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 2   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 1   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 0   |  | 0 |  | 1 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| --- |  |   |  |   |  |   |  |   |  |   |  |   |  |   |  |
|     |  | 0 |  | 1 |  | 2 |  | 3 |  | 4 |  | 5 |  | 6 |  |

... skipping to the last move ...

Current position:

|     |  |   |  |   |  |   |  |   |  |   |  |   |  |   |  |
|-----|--|---|--|---|--|---|--|---|--|---|--|---|--|---|--|
| 5   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 4   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 3   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 2   |  | 0 |  | 0 |  | 0 |  | 2 |  | 0 |  | 0 |  | 0 |  |
| 1   |  | 0 |  | 1 |  | 0 |  | 2 |  | 0 |  | 0 |  | 0 |  |
| 0   |  | 1 |  | 1 |  | 0 |  | 2 |  | 1 |  | 0 |  | 0 |  |
| --- |  |   |  |   |  |   |  |   |  |   |  |   |  |   |  |
|     |  | 0 |  | 1 |  | 2 |  | 3 |  | 4 |  | 5 |  | 6 |  |

It is player 2's move

Arthur, which column (0-6)? *3*

Move 8: player 2 moved in column 3

Current position:

|     |  |   |  |   |  |   |  |   |  |   |  |   |  |   |  |
|-----|--|---|--|---|--|---|--|---|--|---|--|---|--|---|--|
| 5   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 4   |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  | 0 |  |
| 3   |  | 0 |  | 0 |  | 0 |  | 2 |  | 0 |  | 0 |  | 0 |  |
| 2   |  | 0 |  | 0 |  | 0 |  | 2 |  | 0 |  | 0 |  | 0 |  |
| 1   |  | 0 |  | 1 |  | 0 |  | 2 |  | 0 |  | 0 |  | 0 |  |
| 0   |  | 1 |  | 1 |  | 0 |  | 2 |  | 1 |  | 0 |  | 0 |  |
| --- |  |   |  |   |  |   |  |   |  |   |  |   |  |   |  |
|     |  | 0 |  | 1 |  | 2 |  | 3 |  | 4 |  | 5 |  | 6 |  |

Player 2 (Arthur) wins!